

io.them

Strategic Audit & Operating Protocol

NORTH STAR

io.them is a mobile-first AI screenplay studio built around a living creative companion. It helps writers turn voice, fragments, and emotional impulses into properly formatted scenes — fast — while learning their style, characters, tone, and creative habits over time.

Four pillars (every task must serve at least one): 1. Mobile-first · 2. Voice → fragments → formatted scenes · 3. Living creative companion · 4. Longitudinal learning.

A working brief covering: a strategic systems audit of the application, an effort estimate to remediate the audit, and the operating system that lets Codex (primary), Claude (scoped support), and the human (product lead) ship together without collision.

Prepared for

io.them — Engineering & Product

Date

2026-05-09

Contents

Section	Page
1. Strategic Audit — io.them	3
I. Executive Summary	3
II. Critical System Deficiencies	3
III. Strategic Risks	4
IV. Competitive Weaknesses	4
V. User Experience Failures	5
VI. Engineering Concerns	5
VII. AI / Automation Gaps	6
VIII. Recommendations Ranked by Impact	6
IX. Immediate Tactical Actions — next 24 hours	7
X. Long-Term Strategic Directives	7
2. Timeline to Complete	8
Classes of work	8
Engineering effort, by recommendation	8
Aggregate, hard constraints, recommended sequencing	9
3. Operating System	10
Protocol	10
Roles	11
The three coordination files	11
Task discipline & branches	12
Verification & decisions	12
Worktrees & hard scope enforcement	13
Removal principle	14

1. Strategic Audit — io.them

Operational audit of the application *io.them*. The tone is precise and unemotional. The purpose is to surface what is true, ranked by leverage. Each section is structured so the highest-impact items appear first.

I. Executive Summary

The vessel is structurally sound but unevenly engineered. The backend exhibits adult discipline — Postgres domain store, Redis-backed scale backplane, security audit logs with HMAC, typed 429 envelopes, refresh-token replay detection, prompt-regression evals, RC dashboard validation. This is uncommon, and laudable.

The client, by contrast, is a 93,620-line SwiftUI monolith of 220 files in a single flat directory, dominated by a single **ContentView.swift** of 8,000 lines and a **ScreenplayStudioScreen.swift** of 5,903 lines, supported by 62 **DraftStudio*.swift** and 24 **ScreenplayStudio*.swift** siblings. The product surface — a voice-orchestrated screenplay studio with a “Her”-style creative companion (**HerOrb***, **HerVoiceController**, **HerEvolutionStore**, **HerMicroInitiations**) — is ambitious and emotionally distinctive. The execution architecture is not commensurate with that ambition.

The current trajectory is, in a word, *suboptimal*. The product can ship. It cannot, without intervention, scale — neither in users nor in engineers.

II. Critical System Deficiencies

1. **Client-side architectural entropy.** A flat 220-file directory with eight-thousand-line view files is not architecture; it is sediment. Compile times, merge conflicts, and onboarding cost rise super-linearly.
2. **No clear feature boundary.** **DraftStudio***, **ScreenplayStudio***, and **ScreenplayStudioScreen+*** extensions overlap in domain. The naming reveals fission, not modularization.
3. **Debug surface bleeding into production.** Eleven ***Debug*.swift** files at the top level. Useful in development; structurally dangerous shipped.
4. **Two parallel studios.** “DraftStudio” and “ScreenplayStudio” appear to be siblings rather than one workflow. Either consolidate or document the bifurcation. Ambiguity at the product core is the most expensive ambiguity.
5. **Single-process state explosion.** ~30 distinct ***Store.swift** types. No visible orchestrator. Coordination is implicit. Implicit coordination is the origin of most defects.

III. Strategic Risks

- **Founder-key engineering.** A codebase shaped this way can be advanced rapidly only by its author. The bus factor approaches one.

- **App Store reviewer surface.** `test.mp3`, `test.wav`, and `tmp/` inside the app root present an avoidable rejection vector.
- **Identity dilution.** “Framehouse”, “them”, “io.them”, “Clementine”, “Her*” compete for one app. *(Resolved by D001 — canonical name is io.them.)*
- **Voice-pipeline single-vendor exposure.** The realtime stack and the gate's reliance on `OPENAI_API_KEY` indicate hard coupling. Single-supplier outage degrades the entire experience.

IV. Competitive Weaknesses

- **Positioning is implicit, not declared.** *(Resolved by D002 — north star statement is now canonical and printed on the cover.)*
- **The “companion” is the differentiator. It is not yet the headline.** `HerMicroInitiations`, `HerEvolutionStore`, `CreativeCompanionSignals` describe an evolving creative partner. It is buried in 220 sibling files.
- **Mobile-first claim, desktop-shaped surface.**
`DraftStudioCursorInsertTextEditorIOS.swift` as a separate file from the desktop variant suggests parity rather than primacy.

V. User Experience Failures

- **No instant-value moment is visible in the source structure.** Users should hear the orb speak, see one line written, and feel ownership before the third tap.
- **Inconsistent visual vocabulary.** `HerColors.swift`, `FountainTypography.swift`, and assorted `*Chrome*` files imply at least three design systems coexisting.
- **Cognitive load on the studio surface.** Beat rail, outline rail, revision rail, voice-pin rail, assistant rail, scene navigator chrome, mode bar, inspector router, paginated rail — counted from filenames alone. Users desire a single completed scene.
- **Latency theater absent.** No skeletons, optimistic-UI patterns, or perceived-speed scaffolding. Even 300 ms of unhidden latency damages premium perception.

VI. Engineering Concerns

- **File granularity inverted.** Sixty-two small `DraftStudio*` files alongside an eight-thousand-line view file. The correct ratio is the inverse.
- **No visible module boundaries.** No SwiftPM local packages. All compilation is global.
- **Test asymmetry.** The backend has `eval:gate`, `eval:speculative-reuse`, `smoke`, and contract suites. The client surface is comparatively under-instrumented.
- **Persistence sprawl.** JSON stores at backend root coexisting with Postgres. `Decide`. Dual-write is a defect generator.

- **Quality-gate coverage is opt-in by default.** For a release candidate this should be opt-out. The default is the policy.

VII. AI / Automation Gaps

- **The system generates. It does not yet decide.** A creative companion that merely produces text is a 2024 product. A companion that proposes the next beat and reshapes the workspace ahead of the request is a 2026 product.
- **Prompt construction localized, not centralized.** `ScreenplayPromptBuilder.swift` is correct in principle. Verify it is the only path. Decentralized prompts metastasize.
- **No visible memory tier.** The companion must produce a longitudinal user model — preferred tone, recurring characters, structural habits. Without this, the companion is amnesiac.
- **No automation of the dull.** Auto-formatting, auto-naming of scenes, auto-pagination repair, auto-dialogue attribution should be invisible defaults, not features.

VIII. Recommendations Ranked by Impact

#	Recommendation	Effect
1	Decompose the monolith. Extract DraftStudio and ScreenplayStudio into local SwiftPM packages. Reduce ContentView.swift below 500 lines.	Build speed, contributor onboarding, defect rate. Highest leverage.
2	Declare the product in one sentence and rename consistently.	Marketing, App Store conversion, recruiter signal. (Done — D001/D002.)
3	Engineer a 60-second magic moment.	Activation rate.
4	Promote RUN_QUALITY_GATE=1 to default for release-preflight.	A gate that is optional is not a gate.
5	Centralize prompt assembly, add a memory tier, route all generation through it.	Companion coherence, prompt-cost reduction, regression catchability.
6	Adopt perceived-speed primitives system-wide.	Premium perception.
7	Resolve dual persistence. Pick Postgres or JSON. Not both.	Defect reduction.
8	Codify a single design system.	Trust, consistency.
9	Add a second realtime supplier behind the existing interface.	Single-vendor risk mitigation.
10	Instrument the client. Activation, time-to-first-page, retention D1/D7/D30, voice-session length.	You cannot improve what you do not measure.

IX. Immediate Tactical Actions — next 24 hours

- Remove `test.mp3`, `test.wav`, and any non-bundled `tmp/` artifacts from the app target.
- Flip release-preflight to `RUN_QUALITY_GATE=1` by default.
- Identify the three largest extractable subviews of `ContentView.swift`.
- Apply the canonical name `io.them` across README, Info.plist, onboarding copy.
- Add a single client telemetry event: `first_page_written`.

X. Long-Term Strategic Directives

- **Within 30 days:** Modularize into packages (Studio, Companion, Realtime, Persistence, DesignSystem).
- **Within 60 days:** Ship a memory-aware companion. The orb that remembers your voice, your protagonist, and your last unfinished scene is your moat.
- **Within 90 days:** Mobile-primary redesign. Not “responsive.” Native-first.
- **Within 180 days:** Two-supplier realtime. A second voice path. A second model path.
- **Continuously:** Subtraction discipline. For every feature added, identify the feature it replaces.

The probability of user abandonment, given the present friction surface, increases substantially. The probability of engineering velocity collapse, given the present file topology, increases asymptotically with team size. Both are addressable. Neither is addressable later than now.

The companion thesis is sound. The backend hygiene is creditable. The client requires discipline rather than reinvention. The asset is salvageable and, with intervention, formidable.

“The most advanced systems appear simple. This is not accidental. It is the result of disciplined logic.”

Assessment complete.

2. Timeline to Complete

A useful estimate distinguishes three classes of work. Confusing them is the most common cause of bad estimates.

Classes of work

Class	Bottleneck	Notes
Mechanical	Claude / Codex	File extraction, renames, config flips, telemetry plumbing, gate defaults, secondary supplier interface, dead-asset removal.
Judgment-dependent	Human (product lead)	60-second magic moment, design system rules, mobile-primary IA, what to subtract. Agents draft; the human decides via DECISIONS.md.
Externally gated	Reality	App Store review, supplier sign-up, real-device testing, user-validation cycles. No amount of compute compresses these.

Engineering effort, by recommendation

#	Item	Active hrs	Wall-clock
Tac.	Strip test assets, flip RUN_QUALITY_GATE, file extraction tickets, apply canonical name, add first_page_written	2–4	1 day
1	Decompose monolith into SwiftPM packages; reduce ContentView.swift below 500 lines	60–100	2–3 weeks
2	Single product identity end-to-end	3–6	1 day (excl. App Store Connect)
3	60-second magic-moment onboarding	12–20	3–5 days, plus iteration
4	Quality-gate default flip	0.25	30 minutes
5	Centralized prompt assembly + memory tier	24–40	1–2 weeks
6	Perceived-speed primitives system-wide	12–20	3–5 days
7	Resolve dual persistence (Postgres canonical)	12–24	3–6 days
8	Codified design system + lint	16–30	1 week
9	Second realtime supplier behind interface	12–20	3–5 days
10	Client telemetry foundation	6–12	2–3 days

#	Item	Active hrs	Wall-clock
60-day	Memory-aware companion (depth on #5)	30–60	2–3 weeks
90-day	Mobile-primary redesign	60–120	4–8 weeks, iteration-bound
180-day	Two-supplier resilience hardening (depth on #9)	20–40	1–2 weeks

Aggregate

- **Engineering-only, agents on keys:** roughly **270–500 active hours**.
- **Realistic wall-clock with daily collaboration:** **8–14 weeks** through the 90-day directives. The 180-day items push to **~16 weeks** total.
- **If the human responds only weekly:** double the wall-clock.
- **Tactical + 30-day directives only:** **~2–3 weeks**.

Hard constraints we cannot compress

- Agents cannot run the iOS app on a device. The human must verify visual and audio behavior.
- Apple's review cycle is exogenous: typically 24–72 hours per submission.
- The mobile-primary redesign and the 60-second magic moment are *iteration-bound*, not *throughput-bound*.
- Memory-tier work requires real user data to tune. The first version ships in days. The version that *feels* coherent takes weeks of observed use.

Recommended sequencing

1. **Day 1.** Tactical actions. Quality-gate default. Telemetry skeleton. Identity applied (*D001 already accepted*).
2. **Week 1–3.** Modularization. Without it every subsequent change costs more than it should.
3. **Week 3–5.** Centralized prompts + first memory tier + perceived-speed primitives, in parallel.
4. **Week 4–6.** 60-second magic moment built atop the new primitives.
5. **Week 6–8.** Design-system codification, persistence resolution, second supplier.
6. **Week 8–14.** Mobile-primary redesign and memory-tier maturation.

Doing the 60-second moment before modularization is, predictably, a trap. It looks like the highest-leverage work. It is not, until the foundation can support iteration speed.

The single largest determinant of total wall-clock is not agent speed. It is the latency of human decisions. If the product lead pre-commits to: *the workflow that defines success, the persistence canonical, and the design-system rules*, the schedule above holds.

3. Operating System

This section is the team's working contract. It describes how Codex (primary), Claude (scoped support), and the human (product lead) ship together without collision. The contents of this section are mirrored on disk in three files at the repo root: `AGENTS.md`, `TASKS.md`, and `DECISIONS.md`.

Protocol

The multi-agent system exists to **increase velocity, not complexity**. Codex is the primary app builder. Claude is a scoped support agent. The human remains the product lead.

All work flows through `TASKS.md`. No agent starts work without a task row. Each task has **one owner, one branch, one scope, and one definition of done**.

Codex owns the iOS app, product implementation, architecture, and integration. Claude owns backend support, scripts, tests, documentation, audits, and CI work when assigned. Claude does not edit the iOS app or make product decisions.

The team maintains three coordination files: **AGENTS.md** for rules, **TASKS.md** for active work, and **DECISIONS.md** for product and architecture decisions. These files are the operating system of the project.

The purpose of this protocol is to protect the product's central magic: a mobile-first creative companion that helps the user write a scene quickly, emotionally, and beautifully. **Any process that does not help that goal should be removed.**

Roles

	Human	Codex	Claude
Title	Product lead	Primary app builder	Scoped support agent
Owns	Direction. Merges. <code>DECISIONS.md</code> acceptance.	<code>them/</code> , product implementation, architecture, integration.	<code>backend/</code> , scripts, tests, docs, audits, CI when assigned.
Decides	What ships. What doesn't.	How to build what was decided.	Nothing about product. Backend implementation details only.
Edits	Anything.	<code>them/</code> , integration glue, trivial backend tweaks.	<code>backend/</code> , <code>scripts/</code> , <code>tools/</code> , <code>docs/</code> , <code>themTests/</code> , <code>.github/workflows/</code> when assigned.

	Human	Codex	Claude
Never	—	Pushes to main. Merges own PRs.	Edits Swift. Merges PRs. Authors decisions.

The three coordination files

File	Purpose	Who edits
AGENTS.md	The rules. North star, protocol, scope by path, branches, status vocabulary, verification, decision protocol, removal principle.	Human (primary). Codex may propose via PR.
TASKS.md	The active work. One row per task: owner, branch, pillar, status, done-when. The single funnel for all work.	All three. Each agent edits only its own rows.
DECISIONS.md	Product and architecture decisions in ADR form. Accepted decisions are durable; superseded decisions reference the entry that replaced them.	Human accepts. Codex proposes. Claude flags but does not author.

CLAUDE.md exists at the repo root only because Claude Code reads it by convention. It is a thin redirect to AGENTS.md and is not part of the operating system.

Task discipline

- No work without a row in TASKS.md.
- Every row declares owner, branch, scope (in the title), and definition of done.
- An agent claims a row by editing it to Status=in-progress as *the first commit on its new branch*. Conflicts in the ledger surface conflicts in intent before code is touched.
- One owner, one branch, one scope. Splits create new rows.
- PRs reference the task ID. The human merges. Agents do not merge.

Branches

```
codex/<task-id>-<short-name>    # Codex
claude/backend-<short-name>      # Claude
# Never push to main.
```

Status vocabulary (in TASKS.md)

ready → ready-for-claude → in-progress → review → merged

Verification

- **Codex:** iOS build green; themTests pass.

- **Claude:** `cd backend && npm test; npm run eval:gate; RUN_QUALITY_GATE=1 ./scripts/quality_gate.sh` when backend or CI is touched.
- Both agents report what was run and what was not. Never claim verification you skipped.

Decisions

Architectural or product decisions are appended to `DECISIONS.md` in ADR form: id, date, status, context, decision, consequences. Codex may propose. The human accepts. Claude may flag the need for a decision but does not author one.

Decisions already accepted under this protocol:

- **D001** — `io.them` is the canonical product name.
- **D002** — North-star statement (printed on the cover of this document).
- **D003** — Codex-primary, Claude-scoped, human product-lead. (Replaces the earlier two-file protocol.)

Worktrees — physical isolation

Each agent operates in a separate working directory pointing at the same repo. The filesystem prevents collisions. Codex stays in the main checkout; Claude operates in a long-lived worktree.

```
# One-time
mkdir -p .agents/worktrees
echo ".agents/worktrees/" >> .gitignore
git add .gitignore && git commit -m "chore: ignore agent worktrees"

# Claude's persistent worktree
git worktree add .agents/worktrees/claude-backend \
    -b claude/backend-shell main

# Switching to a new task
cd .agents/worktrees/claude-backend
git fetch origin && git checkout main && git pull
git checkout -b claude/backend-<next-task>
```

Hard scope enforcement

Documentation states intent. `.claude/settings.json` enforces it. If Claude tries to edit `them/ContentView.swift`, the harness produces a permission prompt — caught before the change lands.

```
{
  "permissions": {
    "deny": [
      "Edit(them/**)", "Write(them/**)",
      "Edit(them.xcodeproj/**)", "Write(them.xcodeproj/**)",
      "Edit(them.xcworkspace/**)", "Write(them.xcworkspace/**)",
      "Edit(AGENTS.md)", "Write(AGENTS.md)",
      "Edit(DECISIONS.md)", "Write(DECISIONS.md)",
      "Edit(Archive/**)", "Edit(Library/**)", "Edit(Projects/**)",
      "Edit(**/Info-*.plist)",
    ]
  }
}
```

```

    "Edit(**/*.entitlements)",
    "Edit(**/PrivacyInfo.xcprivacy)",
    "Edit(backend/*_store.json)",
    "Edit(backend/**/knowledge_embeddings_cache.json)"
  ],
  "allow": [
    "Bash(cd backend && npm *)",
    "Bash(./scripts/quality_gate.sh*)",
    "Bash(RUN_QUALITY_GATE=1 ./scripts/quality_gate.sh*)"
  ]
}
}

```

Removal principle

Any process, file, or rule that does not help the user write a scene quickly, emotionally, and beautifully is a candidate for deletion. Review the operating system quarterly. Subtract before adding.

Closing

The operating system is small on purpose. Three files. One funnel. One product lead, one primary builder, one scoped supporter. Anything more elaborate would violate the first sentence of the protocol — that the system exists to increase velocity, not complexity. Install it. Edit it as reality teaches you. Remove what stops earning its place.

End of brief.